

Heap Memory Management

- Heap is used to allocate space for objects created at run time.
 - For example: nodes of dynamic data structures such as linked lists and trees.
- Dynamic memory allocation and deallocation are based on the requirements of the program.
 - *malloc()* and *free()* in C programs.
 - *new()* and *delete()* in C++ programs.
 - *new()* and garbage collection in Java programs.
- Allocation and deallocation may be *completely manual* (C/C++), *semi-automatic* (Java), or *fully automatic* (Python).



Memory Manager

- Manages heap memory by implementing mechanisms for allocation and deallocation, both manual and automatic.
- Goals
 - Space efficiency: minimize fragmentation.
 - Program efficiency: take advantage of locality of objects in memory and make the program run faster.
 - Low overhead: allocation and deallocation must be efficient.
- Heap is maintained either as a *doubly linked list* or as *bins* of free memory chunks (more on this later).

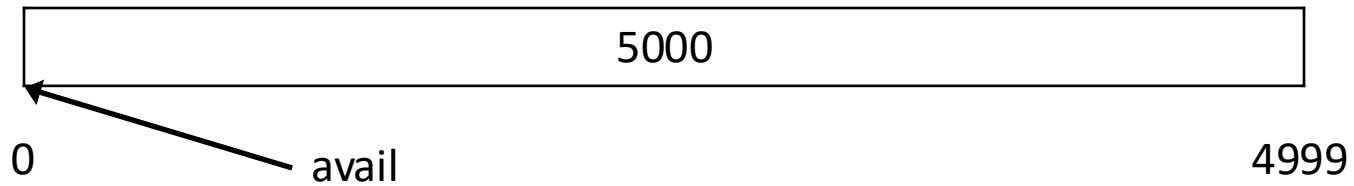


Allocation and Deallocation

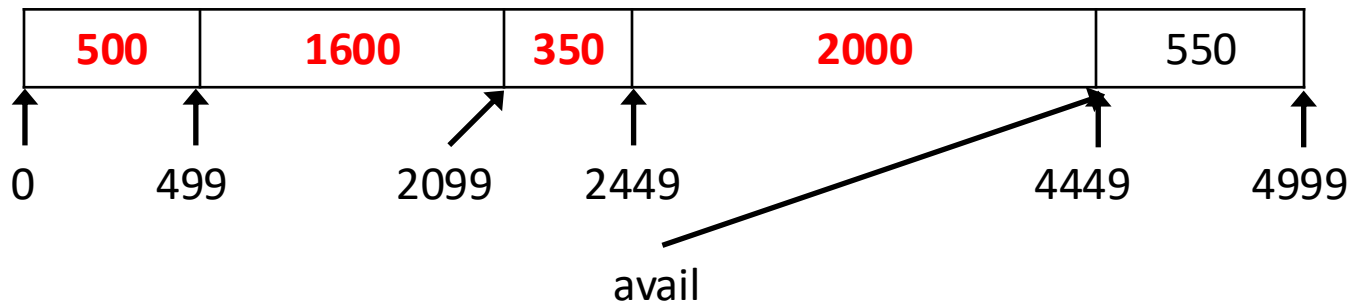
- In the beginning, the heap is one large and contiguous block of memory.
- As allocation requests are satisfied, chunks are cut off from this block and given to the program.
- As deallocations are made, chunks are returned to the heap and are free to be allocated again (*holes*).
- After several allocations and deallocations, heap is fragmented and not contiguous.
- Allocation from a fragmented heap may be made either in a *first-fit* or *best-fit* manner.
- After a deallocation, we try to *coalesce* contiguous holes and make a bigger hole (free chunk).



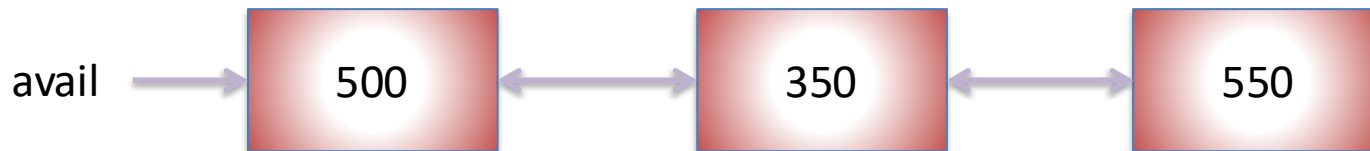
Initial
heap



Heap after allocations (red) of 500, 1600, 350, and 2000 size blocks.



Heap after blocks of size 500 and 350 are freed. Blocks of size 1600 and 2000 are still occupied.



First-Fit and Best-Fit Allocation Strategies

- The *first-fit* strategy picks the **first** available chunk that satisfies the allocation request.
- The *best-fit* strategy searches and picks the smallest (**best**) available chunk that satisfies the allocation request.
- Both chop off a block of the required size from the chosen chunk and return it to the program.
- The rest of the chosen chunk remains in the heap.

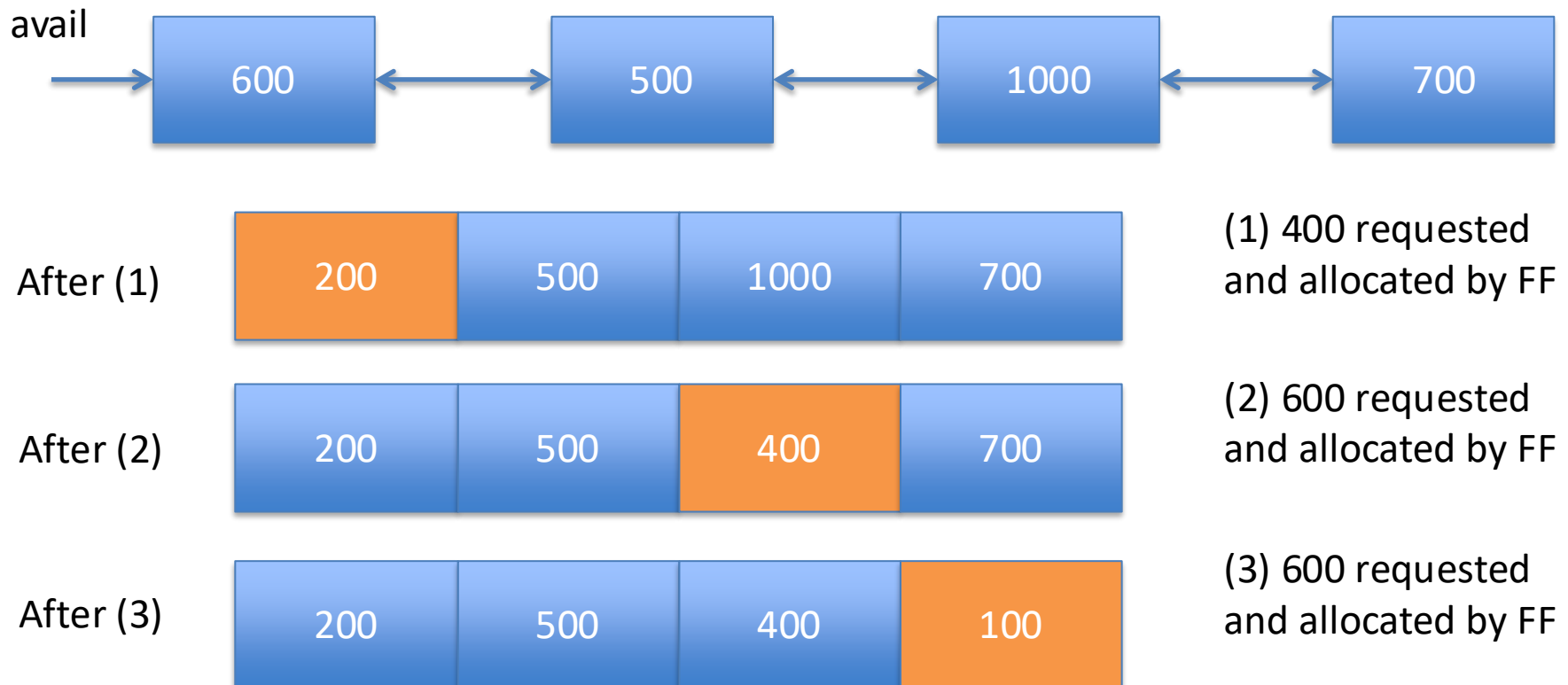


Next-Fit Allocation Strategy

- Best-fit strategy has been shown to reduce fragmentation in practice, better than first-fit strategy.
- But first-fit is much faster than best-fit.
- *Next-fit* strategy tries to allocate the object in the chunk that has been split recently.
 - Tends to improve speed of allocation.
 - Tends to improve spatial locality since objects allocated at about the same time tend to have similar reference patterns and lifetimes (cache behavior may be better).



Example of F-F Allocation Failure and B-F Allocation Success (1)

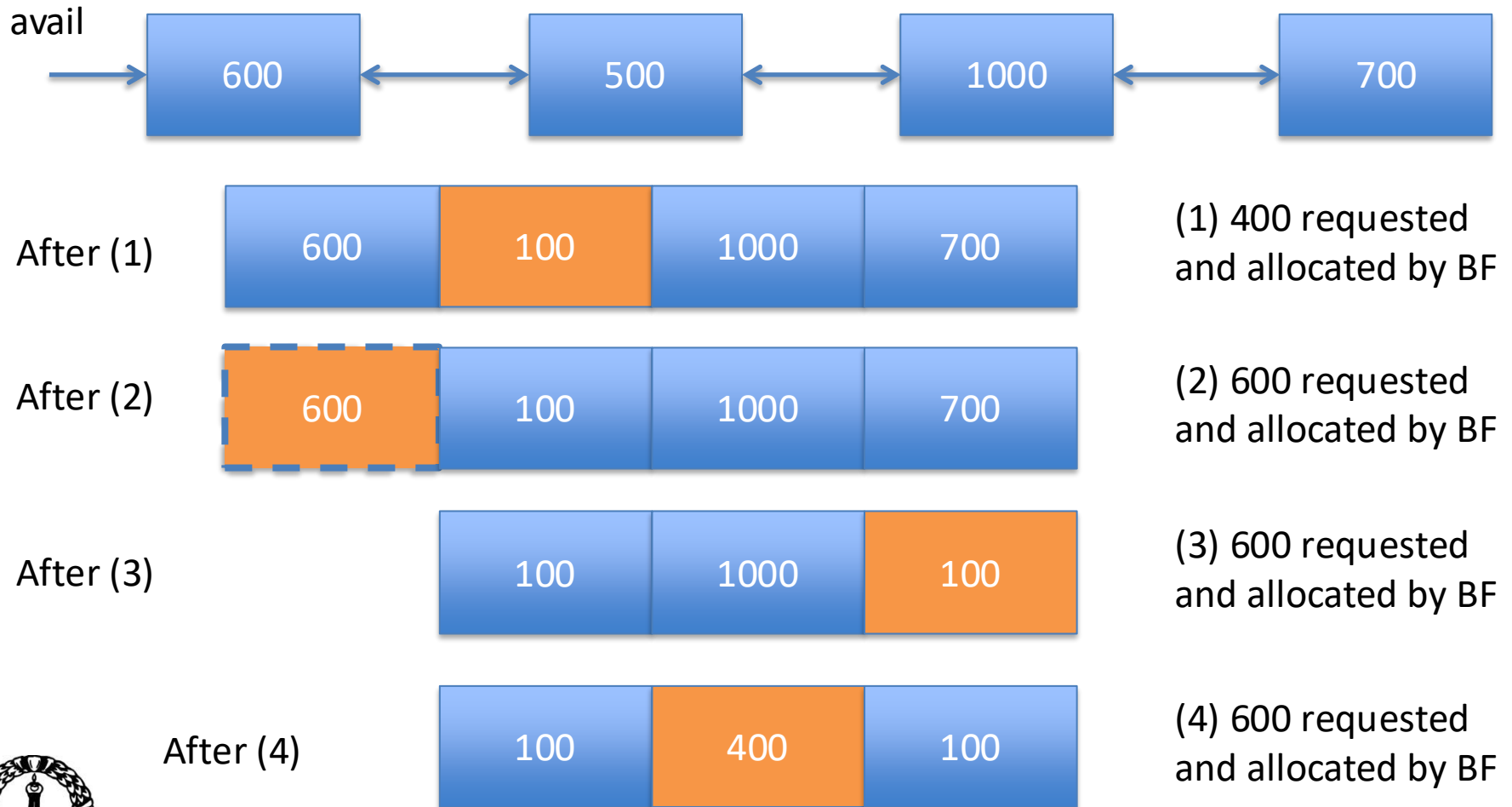


Note: double links have been omitted from the diagrams here and in the following slides to save space.

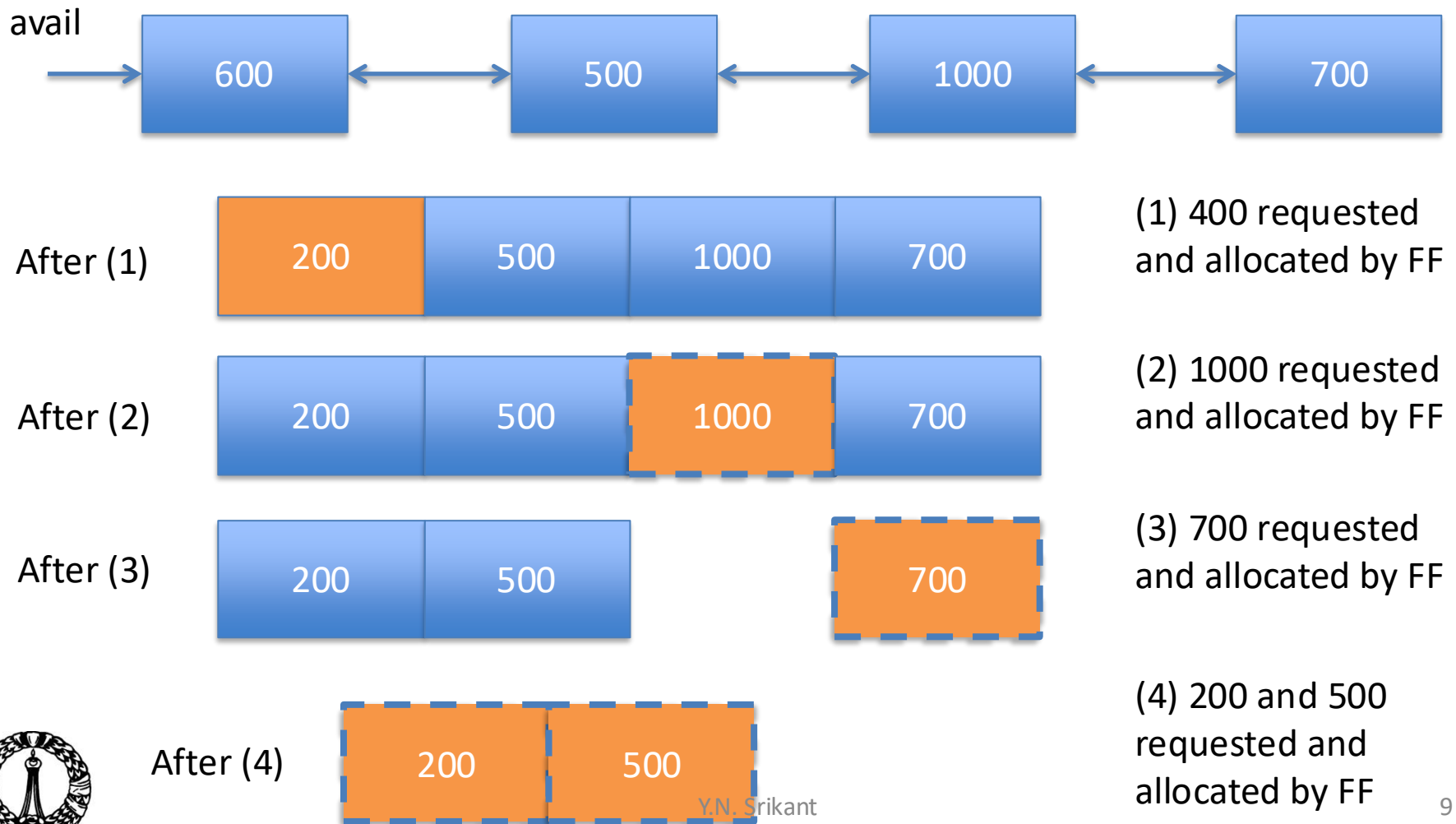
(4) 600 requested and CANNOT BE ALLOCATED by FF



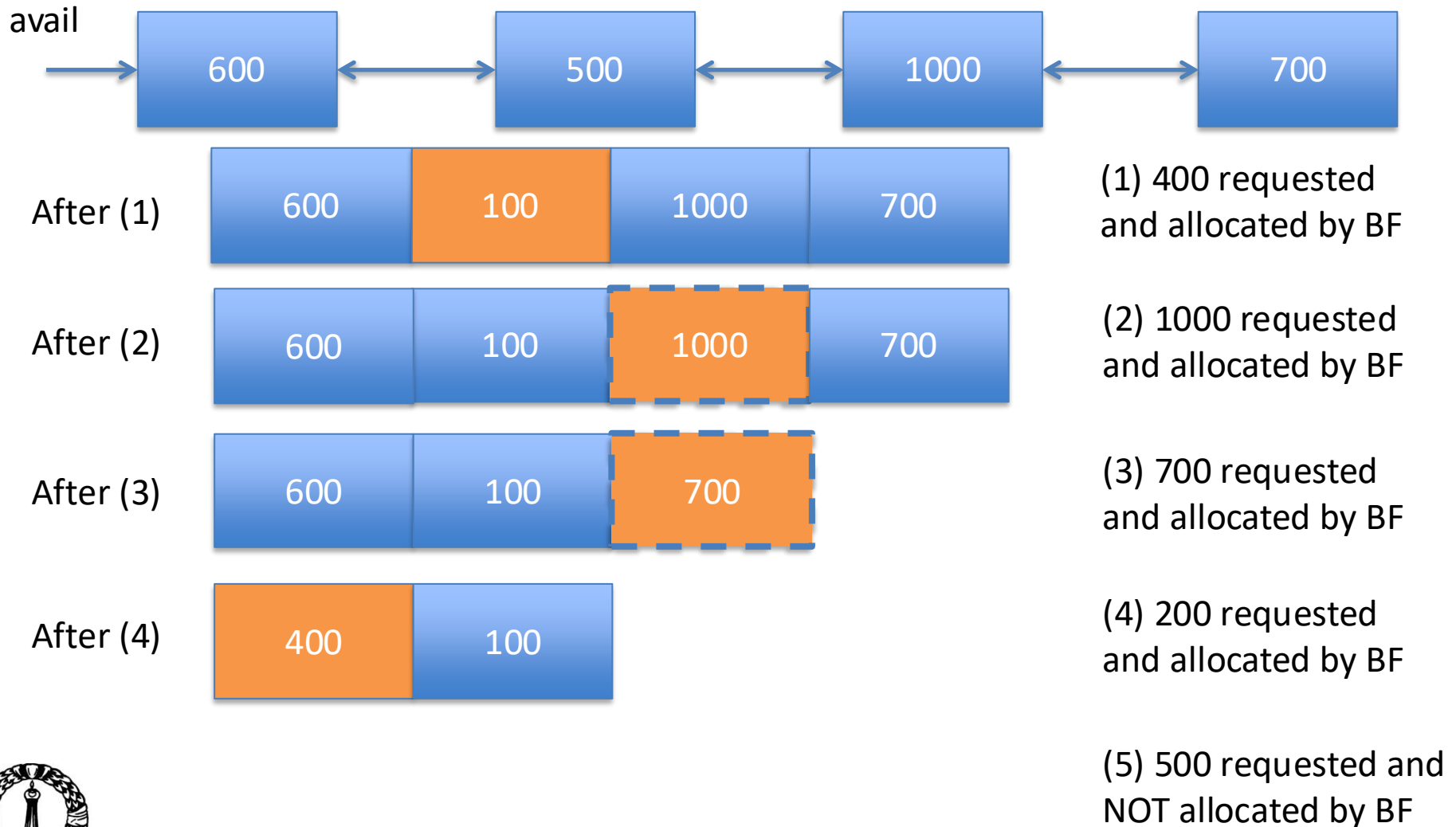
Example of F-F Allocation Failure and B-F Allocation Success (2)



Example of B-F Allocation Failure and F-F Allocation Success (1)



Example of B-F Allocation Failure and F-F Allocation Success (2)

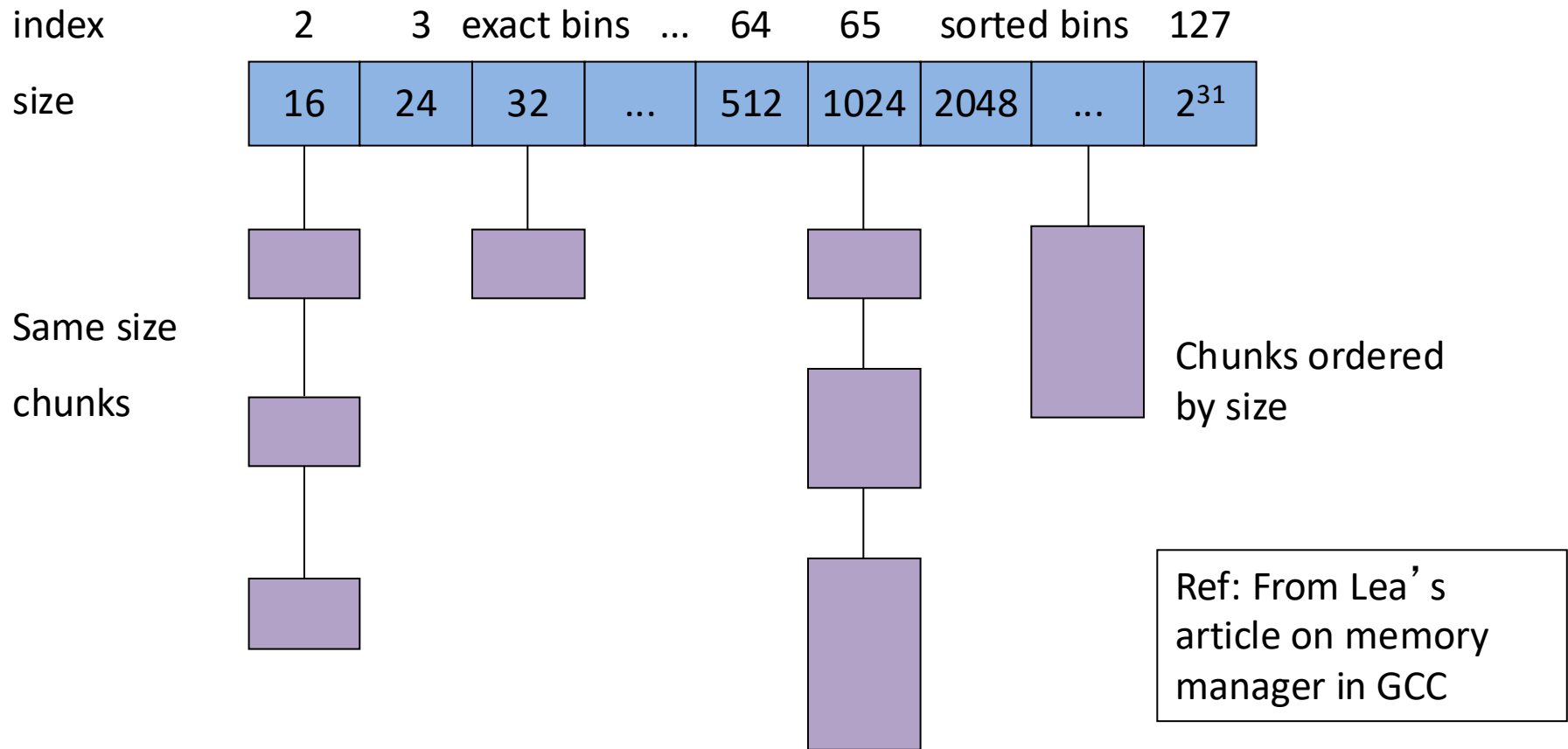


Bin-based Heap

- Free space is organized into *bins* according to their sizes ([Lea Memory Manager in GCC](#)).
 - Many more bins for smaller sizes, because there are many more small objects.
 - A bin for every multiple of 8-byte chunks from 16 bytes to 512 bytes.
 - Then approximately double previous size.
 - Within each “small size bin”, all chunks are of the same size.
 - In others, they are ordered by size.
 - The last chunk in the last bin is the *wilderness chunk*, which gets us a chunk by going to the operating system.



Bin-based Heap – An Example



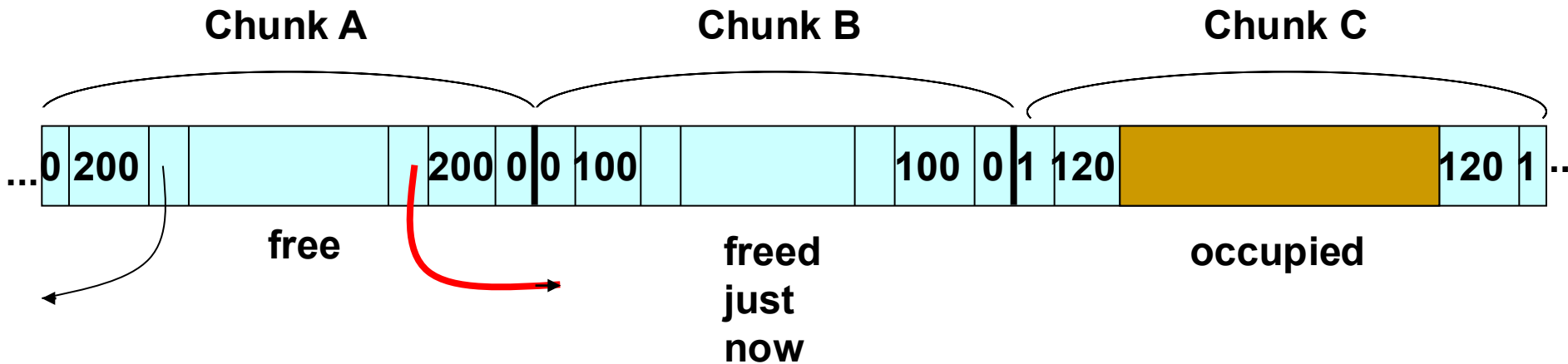
Managing and Coalescing Free Space

- Should coalesce adjacent chunks and reduce fragmentation.
 - Many small chunks together cannot hold one large object.
 - In the **Lea memory manager**, no coalescing in the exact size bins, only in the sorted bins.
 - **Boundary tags** (free/used bit and chunk size) at each end of a chunk (for both used and free chunks).
 - A *doubly linked list* of free chunks.



Boundary Tags and Doubly Linked List

3 adjacent chunks. Chunk B has been freed just now and returned to the free list. Chunks A and B can be merged, and this is done just before inserting it into the linked list. The merged chunk AB may have to be placed in a different bin in Lea's method.



Boundary Tags and Doubly Linked List

3 adjacent chunks. Chunk B has been freed just now and returned to the free list. Chunks A and B can be merged, and this is done just before inserting it into the linked list. The merged chunk AB may have to be placed in a different bin in Lea's method.

